COGNITIVE TWIN & INTERVIEW SYSTEM

An implementation-grounded engineering dossier | First-person architectural account | Aegis Agent

My engineering position. I built the Aegis cognitive-twin foundation around a consented, locally persisted operator profile rather than a claim of autonomous psychological modeling. I use a structured Day-0 interview to capture explicit operator inputs, persist a versioned representation and make that profile available to bounded downstream proposal workflows. Here I distinguish what my public code implements from what it does not establish.

01 | The boundary I designed

I separate interview capture, profile construction, proposal generation and human authorization. A stored profile is context for a later proposal; it is not permission to perform an action. The default Aegis deployment is local-first, with SQLite state under AEGIS_DATA_DIR and offline Echo inference unless an optional adapter is configured.

Source surface	Responsibility
core/twin_interview.py	Ordered six-question session, durable answers, consent-gated commit and latest-profile retrieval.
core/twin_schema.py	Typed six-layer profile, version metadata and deterministic SHA-256 fingerprint.
core/twin_actions.py	Profile-dependent local proposals, persisted action states and separate approval/execution path.
core/twin_risk.py	Heuristic action risk labels; not an autonomous permission decision.
core/twin_memory_control.py	Tenant-scoped memory visibility, field suppression and deletion-related operations.

02 | My six-layer Day-0 interview

I use a fixed, ordered question bank. The session tracks session_id, tenant_id, answers, current_index, complete and the next expected question. Each accepted answer advances the index and persists the new state. Submitting an unexpected question ID raises an error rather than silently reordering the interview.

Interview dimension	Captured operator input
Role	Primary professional role
Decision style	Approach to decisions under uncertainty
Tools	Frequently used tools and platforms
Risk posture	Self-described tolerance and approach to risk
Work ethics	Operator-stated working principles
Repositories	Projects or repositories under responsibility

03 | The session lifecycle I can substantiate

Start. I generate a session identifier, associate it with a tenant and persist an initial state with the first question. Answer. I accept the current expected question and store its text. Complete. After six answers, I mark the interview complete. Commit. I require explicit consent and a completed interview before creating a versioned profile. Read. I retrieve the latest committed profile for the specified tenant.

The last_tenant file is only a local convenience marker for the most recently active tenant; it is not the authoritative profile record or an authentication mechanism.

04 | Consent is a persistence gate, not a blanket mandate

In TwinInterviewStore.commit, I reject consent=False with a permission error and reject incomplete interviews. I then construct a TwinProfile from the six submitted answers and persist it to SQLite. The commit operation stores a consent flag with the profile. I do not interpret this single flag as proof of legally sufficient consent for every downstream use, external transfer or future product integration.

Case	Observed code-level outcome
Consent absent	Profile commit denied.
Interview incomplete	Profile commit denied.
Unknown session	Session lookup fails with a typed error.
Out-of-order answer	Unexpected question identifier rejected.
Completed session + consent	Versioned profile constructed and persisted.

05 | Versioning and profile integrity

I assign the next profile version per tenant by querying the maximum stored version plus one. Each committed profile receives its own profile identifier and timestamps. I calculate a SHA-256 fingerprint over canonical JSON containing only the six layer values; timestamps are intentionally excluded. This makes the fingerprint deterministic for identical layer content.

A fingerprint is a content-comparison primitive, not a digital signature. It does not authenticate the operator or protect against a party able to rewrite both the stored profile and its fingerprint. I do not describe it as cryptographic identity verification.

06 | How the profile enters the proposal workflow

In propose_actions(tenant_id), I require an existing consented profile. The deterministic baseline proposals include review_digest and prepare_weekly_plan; a review_repos proposal is added when repository input is present. The proposal records carry a tenant identifier and risk classification. This is a bounded template-driven use of profile context, not evidence that a model has learned a person's behavior autonomously.

Layer	Concrete boundary
Interview	Captures explicit answers; does not perform actions.

Layer	Concrete boundary
Profile	Stores six values and metadata in SQLite.
Proposal	Generates bounded suggested work items from a consented profile.
Approval	Separate operator decision and action digest binding.
Execution	Local approved effects only; no general autonomous external execution.

07 | Tenant scope and durable state

I query profile versions by tenant and return the latest committed profile for that tenant. Interview sessions and profile records are persisted in SQLite, so they can survive process restart when the same data directory is reused. I also expose tenant-scoped memory-control operations. These code paths demonstrate logical tenant filtering; they do not, by themselves, prove a hosted multi-tenant isolation perimeter or independently authenticated tenant identity.

The default data-at-rest configuration is not automatically encrypted. Optional encryption paths exist elsewhere in the repository, but I do not imply that every local database is encrypted by default.

08 | My privacy and deletion qualifications

The memory-control module includes visible-field listings, a forget mechanism and broader tenant-cleanup functions. These operate on specified stores and artifacts. I do not claim that a single call proves universal erasure of every possible backup, archive, exported file or independently copied artifact. A deletion receipt is evidence that a cleanup path ran, not proof that all external copies have ceased to exist.

09 | Failure modes I would test before stronger claims

Scenario	Why it matters
Duplicate or concurrent commits	Version allocation uses a max-plus-one query; this review does not establish a distributed serialization guarantee.
Cross-tenant session identifier use	The answer/commit API accepts session IDs; authentication and caller-to-tenant binding require separate end-to-end verification.
Malformed operator input	Question ordering is checked; comprehensive content validation is a different concern.
Profile tampering	Unkeyed SHA-256 is not an authenticated integrity proof.
Backup and restore	Local backup primitives exist; completeness across every derived artifact needs dedicated verification.

10 | Verification standard and evidence limits

I inspected the actual implementations of core/twin_interview.py, core/twin_schema.py, core/twin_actions.py, core/twin_risk.py, core/twin_memory_control.py and core/twin_backup.py in the public repository. This dossier is a source-level review; I did not run the complete test suite or a live operator interview for this publication. I therefore make no fresh pass-rate, performance, concurrency or production-certification claim.

11 | Claims I make, and claims I deliberately withhold

I can substantiate: a six-question stateful Day-0 interview, consent-gated profile commit, SQLite persistence, per-tenant profile versioning, deterministic content fingerprinting, and a bounded profile-dependent proposal path.

I do not claim: autonomous behavioral learning, psychological inference accuracy, independently verified identity, blanket regulatory compliance, universal data erasure, hosted SaaS tenant isolation or permissionless external action execution.

My architectural conclusion. I treat the cognitive twin as an explicit, inspectable operator-context model. I designed its current value around transparent inputs, persisted state and a human-controlled proposal boundary. More ambitious cognitive adaptation remains a separate engineering question, not a shipped capability asserted by this document.

12 | Concrete engineering scenarios

Scenario A - Interrupted interview. I persist answers after each accepted question. A new store using the same data directory can read the saved session state; I do not promise that every UI automatically resumes without the corresponding session identifier.

Scenario B - Refused consent. Six completed answers alone do not create a committed profile. The commit call requires a positive consent argument, and downstream baseline proposals require a consented profile.

Scenario C - Profile revision. A later completed interview creates a new tenant-scoped version; the fingerprint represents current layer values, not a complete behavioral history.

14 | Public implementation references

Source: github.com/aminazimi42-coder/aegis-agent. Primary files: core/twin_interview.py, core/twin_schema.py, core/twin_actions.py, core/twin_risk.py, core/twin_memory_control.py, core/twin_backup.py, README.md and STATUS.md. I have not changed repository content in preparing this publication.

Authorship: Amin Azimi | AI Architect | Azimi Innovation Lab. All first-person engineering statements describe the inspected public implementation; they are not independent operational certification.